



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №3

з дисципліни «Технології розроблення програмного забезпечення»

Тема: «13. Office communicator»

Виконав:

студент групи ІА-32
Шадлер Андрій

Перевірив:

вик. Мягкий М. Ю.

Тема: Основи проектування розгортання.

Мета роботи: Навчитися проектувати діаграми розгортання та компонентів для системи що проектується, а також розробляти діаграми взаємодії, а саме діаграми послідовностей, на основі сценаріїв зроблених в попередній лабораторній роботі

Зміст

Теоретичні відомості.....	2
Хід виконання:.....	4
Завдання.....	4
Діаграма розгортання системи	5
Діаграма компонентів	7
Діаграма послідовностей	8
Вихідний код	10
Контрольні запитання.....	17
Висновок	18

Теоретичні відомості

Діаграми компонентів — показують модулі (компоненти), їхні залежності і артефакти (.jar, таблиці, html). Використовуються для логічного/фізичного поділу системи.

Діаграма компонентів UML є представленням проєктованої системи, розбитої на окремі модулі. Залежно від способу поділу на модулі розрізняють три види діаграм компонентів:

- логічні;
- фізичні;
- виконувані.

Коли використовують логічне розбиття на компоненти, то у такому разі проєктовану систему віртуально уявляють як набір самостійних, автономних модулів (компонентів), що взаємодіють між собою.

Діаграми розгортання — показують фізичні вузли (devices) і середовища виконання (execution environments), на яких розгортаються артефакти; зв'язки зазначають протоколи (HTTP, SQL/ODBC).

Діаграми розгортання представляють фізичне розташування системи, показуючи, на якому фізичному обладнанні запускається та чи інша складова програмного забезпечення.

Головними елементами діаграми є вузли, пов'язані інформаційними шляхами. Вузол (node) – це те, що може містити програмне забезпечення. Вузли бувають двох типів. Пристрій (device) – це фізичне обладнання: комп'ютер або пристрій, пов'язаний із системою. Середовище виконання (execution environment) – це програмне забезпечення, яке саме може включати інше програмне забезпечення, наприклад операційну систему або процес-контейнер (наприклад, вебсервер).

Діаграми послідовностей — моделюють часову послідовність повідомлень між акторами/об'єктами (HTTP POST/GET: Browser – Server DB – Browser).

Діаграма послідовностей (Sequence Diagram) – це один із типів діаграм у моделюванні UML (Unified Modeling Language), який використовується для моделювання взаємодії між об'єктами системи у певній послідовності часу. Вона відображає, як об'єкти обмінюються повідомленнями, показуючи порядок і логіку виконання операцій. Діаграма складається з таких основних елементів:

Актори (Actors): Зазвичай позначаються піктограмами або назвами. Це користувачі чи інші системи, які взаємодіють із системою. Актори можуть бути

Об'єкти або класи: Розміщуються горизонтально на діаграмі. Вони позначаються прямокутниками з іменем об'єкта або класу під прямокутником.

Кожен об'єкт має «життєвий цикл», який представлений вертикальною пунктирною лінією (лінія життя).

Повідомлення: Це лінії зі стрілками, які з'єднують об'єкти. Вони показують передачу повідомлень чи виклик методів. Стрілка може бути синхронною (звичайна стрілка) або асинхронною (лінія з відкритим трикутником) та з пунктирною лінією, що показує повернення результату.

Активності: Вказують періоди, протягом яких об'єкт виконує певну дію.

На діаграмі це позначається прямокутником, накладеним на лінію життя.

Контрольні структури: Використовуються для відображення умов, циклів або альтернативних сценаріїв. Наприклад, блоки "alt" (альтернатива) або "loop" (цикл).

Хід виконання:

Завдання

- Ознайомитись з короткими теоретичними відомостями.
- Проаналізувати діаграми створені в попередній лабораторній роботі а також тему системи та спроектувати діаграму розгортання використання відповідно до обраної теми лабораторного циклу.
- Розробити діаграму компонентів для проєктованої системи.
- Розробити діаграму розгортання для проєктованої системи.
- Розробити як мінімум дві діаграми послідовностей для сценаріїв прописаних в попередній лабораторній роботі.
- На основі спроектованих діаграм розгортання та компонентів доопрацювати програмну частину системи. Реалізація системи, додатково до попередньої реалізації, повинна містити як мінімум дві візуальні форми. В системі вже повинен бути повністю реалізована архітектура (повний цикл роботи з даними від вводу на формі до збереження їх в БД і подальшій виборці з БД та відображенням на UI).
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму розгортання з описом, діаграму компонентів системи з описом, діаграми послідовностей, а також вихідний код системи, який було додано в цій лабораторній роботі.

Office communicator (strategy, adapter, abstract factory, bridge, composite, client-server)

Мережевий комунікатор для офісу повинен нагадувати функціонал програми Skype з можливостями голосового / відео / конференц-зв'язку, відправки текстових повідомлень і файлів (можливо, оффлайн), веденням організованого списку груп / контактів

Діаграма розгортання системи

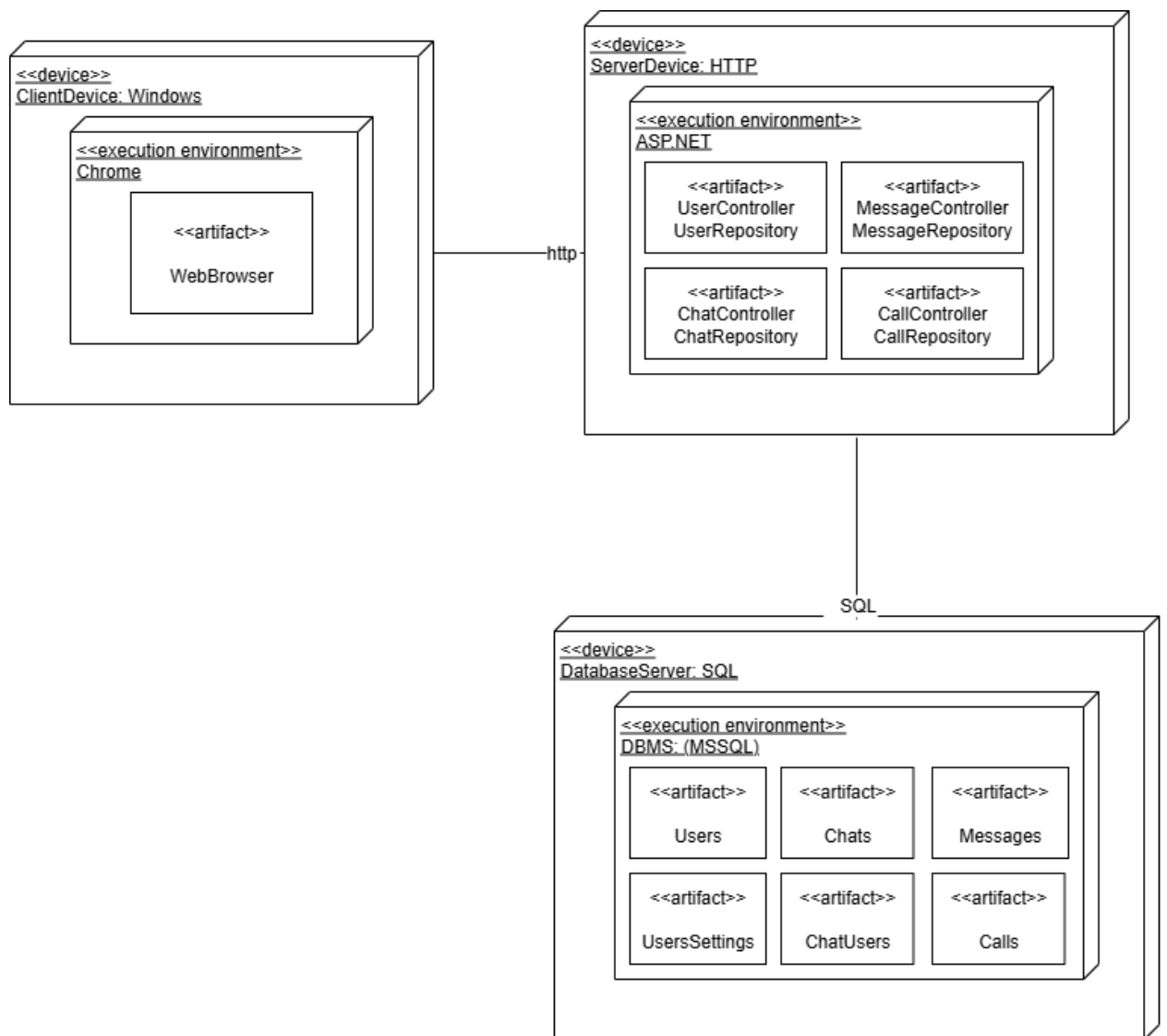


Рис. 1. Діаграма розгортання системи

Опис:

1. ClientDevice: Windows
 - a. Це фізичний пристрій користувача
 - b. Усередині розгорнуто середовище Chrome (<<execution environment>>), яке представляє веб-бравзер
 - c. У бравзері запускається артефает “Web Browser”, який відповідає за інтерфейс та відправку HTTP запитів до серверу

2. ServerDevice: HTTP

- a. Серверна машина, що обробляє HTTP-запити
- b. Виконується ASP.NET, у якому розміщені серверні артефакти
- c. UserController -> UserRepository – модуль, що відповідає за обробку запитів пов'язаних із користувачами (створення, авторизація, зміна налаштувань/профілю)
- d. ChatController -> ChatRepository – модуль, що відповідає за обробку запитів пов'язаних із чатом (створення, відображення повідомлень)
- e. MessageController -> MessageRepository, а також CallController -> CallRepository – модулі, що відповідають за збереження історії повідомлень або дзвінків відповідно

3. DatabaseServer: SQL

- a. Це сервер, що відповідає за збереження та обробку даних
- b. Усередині виконується DBMS (MSSQL) як середовище виконання
- c. Users(Table) – таблиця для збереження користувачів
- d. UsersSettings(Table) – таблиця для збереження налаштувань користувачів
- e. Chats(Table) – таблиця для збереження чатів
- f. ChatUsers(Table) – допоміжна таблиця для розв'язку багатовимірних зв'язків між Users та Chats у випадку групових чатів.
- g. Messages(Table) – таблиця для збереження логів повідомлень
- h. Calls(Table) – таблиця для збереження логів дзвінків

4. Зв'язки:

- a. ClientDevice-ServerDevice – взаємодія через HTTP-протокол, для надсилання та отримання запитів і відповідей
- b. ServerDevice-DatabaseDevice – взаємодія через SQL, для створення нових чи пошуку існуючих записів у базі даних

Діаграма компонентів

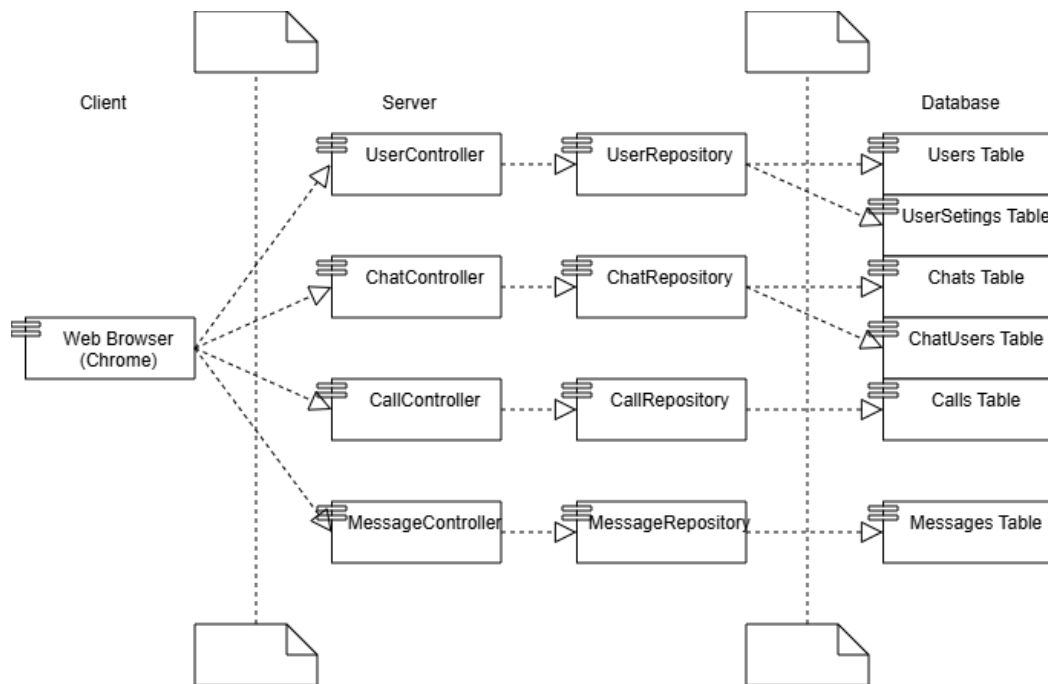


Рис. 2. Діаграма компонентів

Опис:

1. Client(Клієнтська частина):
 - a. Компонент **Web Browser (Chrome)** відповідає за взаємодію з користувачем. Він відправляє HTTP-запити на сервер і відображає UI-відповіді
2. Server(Серверна частина):
 - a. Контролери (**UserController**, **ChatController**, **CallController**, **MessageController**) керують репозиторіями (**UserRepository**, **ChatRepository**, **CallRepository**, **MessageRepository**)
 - b. Репозиторії (**UserRepository**, **ChatRepository**, **CallRepository**, **MessageRepository**) напряму обробляють запити користувачів і надсилають дані в базу даних
3. Database(База даних):
 - a. **Users Table** – таблиця для збереження інформації про клієнтів
 - b. **UserSettings Table** – таблиця для збереження інформації про персоналізацію клієнтського інтерфейсу
 - c. **Chats Table** – таблиця для збереження чатів та їх описів

- d. ChatUsers Table – поміжна таблиця для збереження приналежності користувачів до чатів
 - e. Calls Table – таблиця для збереження записів про дзвінки
 - f. Messages Table – таблиця для збереження повідомлень
4. Зв'язки:
- a. Web Browser надсилає запити до контролерів
 - b. Контролери передають їх у репозиторії
 - c. Репозиторії обробляють запити та надсилають їх у бази даних

Діаграма послідовностей

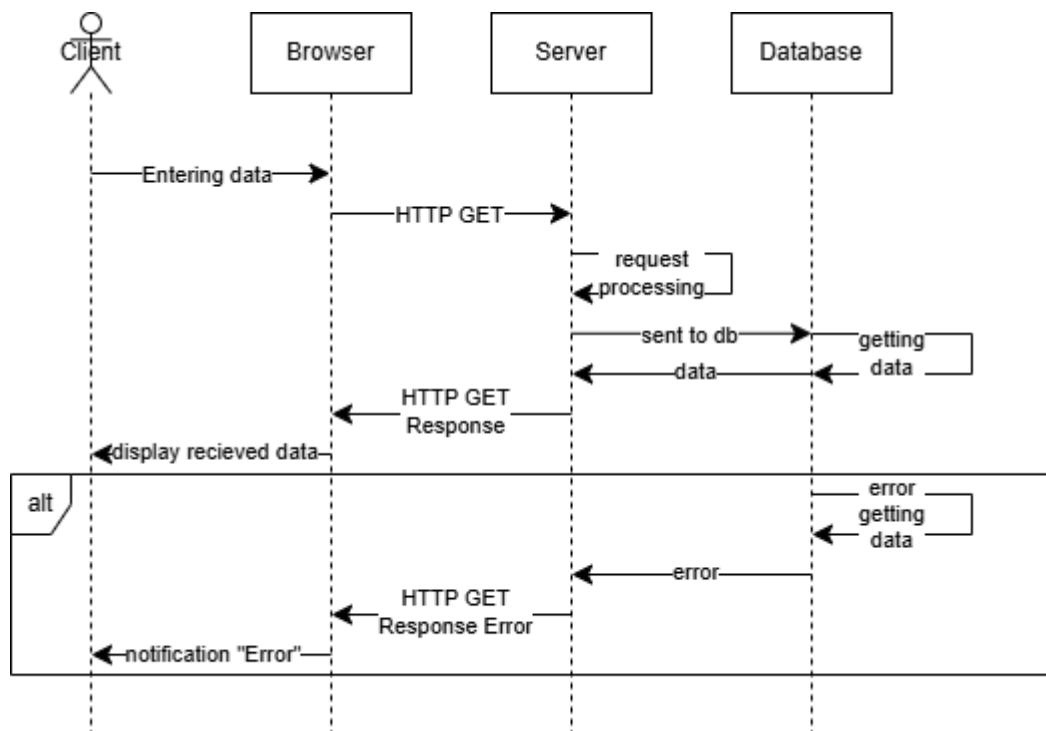


Рис. 3.1. Діаграма послідовності (GET)

Опис:

1. Перебіг подій:
 - a. Клієнт взаємодіє з браузером
 - b. Browser відправляє HTTP GET-запит на Server
 - c. Server звертається до БД за даними
 - d. БД повертає список даних
 - e. Server відправляє дані назад у Browser

- f. Browser відображає їх на UI
- g. У разі винятку – сервер викидає помилку й повертає повідомлення з деталями помилки

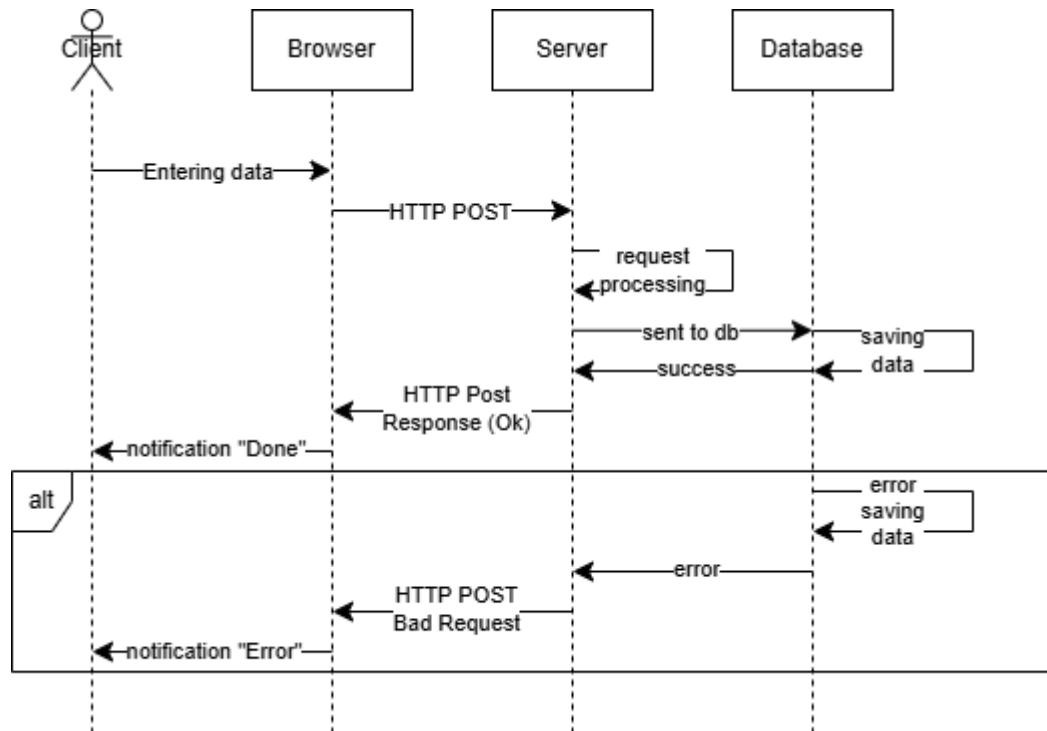


Рис. 3.2. Діаграма послідовності (POST)

Опис:

1. Перебіг подій:
 - a. Клієнт взаємодіє з браузером
 - b. Browser відправляє HTTP POST-запит на Server
 - c. Server звертається до БД і зберігає дані
 - d. БД повертає результат
 - e. Server формує HTTP-відповідь і надсилає у Browser
 - f. Browser показує результат користувачу
 - g. У разі винятку – сервер викидає помилку й повертає повідомлення з деталями помилки

Вихідний код

```
namespace ChatApi.Models;

public interface IChatRecord
{
    public long Id { get; set; }
    public long ChatId { get; set; }
    public DateTime Timestamp { get; set; }
}

namespace ChatApi.Models;

public class User
{
    public long Id { get; set; }
    public string Username { get; set; } = null!;
    public string? DisplayName { get; set; }
    public string? Email { get; set; }
    public string? PasswordHash { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public UserSettings? Settings { get; set; }
    public ICollection<ChatUser>? ChatUsers { get; set; }
    public ICollection<Message>? Messages { get; set; }
    public ICollection<Call>? Calls { get; set; }
}

namespace ChatApi.Models;

public class UserSettings
{
    public long Id { get; set; }
    public long UserId { get; set; }
    public string? PreferencesJson { get; set; }

    public User? User { get; set; }
```

```

    }

namespace ChatApi.Models;

public class Chat
{
    public long Id { get; set; }
    public string? Title { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;

    public ICollection<ChatUser>? ChatUsers { get; set; }
    public ICollection<Message>? Messages { get; set; }
    public ICollection<Call>? Calls { get; set; }
}

namespace ChatApi.Models;

public class ChatUser
{
    public long ChatId { get; set; }
    public long UserId { get; set; }
    public string? Role { get; set; }

    public Chat? Chat { get; set; }
    public User? User { get; set; }
}

namespace ChatApi.Models;

public class Call : IChatRecord
{
    public long Id { get; set; }
    public long ChatId { get; set; }
    public long? InitiatorId { get; set; }
    public DateTime Timestamp { get; set; } = DateTime.UtcNow;
    public DateTime? StartedAt { get; set; }
    public DateTime? EndedAt { get; set; }
}

```

```

public Chat? Chat { get; set; }

public User? Initiator { get; set; }

}

namespace ChatApi.Models;

public enum MessageType
{
    Text = 0,
    System = 1,
    File = 2
}

public class Message : IChatRecord
{
    public long Id { get; set; }
    public long ChatId { get; set; }
    public long SenderId { get; set; }
    public string Content { get; set; } = string.Empty;
    public MessageType Type { get; set; } = MessageType.Text;
    public DateTime Timestamp { get; set; } = DateTime.UtcNow;

    public Chat? Chat { get; set; }
    public User? Sender { get; set; }
}

using ChatApi.Models;
using Microsoft.EntityFrameworkCore;

namespace ChatApi.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<User> Users => Set<User>();
    public DbSet<UserSettings> UsersSettings => Set<UserSettings>();

```

```

public DbSet<Chat> Chats => Set<Chat>();

public DbSet<ChatUser> ChatUsers => Set<ChatUser>();

public DbSet<Call> Calls => Set<Call>();

public DbSet<Message> Messages => Set<Message>();


protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<User>(b =>
    {
        b.HasKey(u => u.Id);
        b.HasIndex(u => u.Username).IsUnique();
        b.HasOne(u => u.Settings).WithOne(s => s.User).HasForeignKey<UserSettings>(s => s.UserId);
    });

    modelBuilder.Entity<UserSettings>(b =>
    {
        b.HasKey(s => s.Id);
    });

    modelBuilder.Entity<Chat>(b =>
    {
        b.HasKey(c => c.Id);
    });

    modelBuilder.Entity<ChatUser>(b =>
    {
        b.HasKey(cu => new { cu.ChatId, cu.UserId });
        b.HasOne(cu => cu.Chat).WithMany(c => c.ChatUsers).HasForeignKey(cu => cu.ChatId);
        b.HasOne(cu => cu.User).WithMany(u => u.ChatUsers).HasForeignKey(cu => cu.UserId);
    });

    modelBuilder.Entity<Message>(b =>
    {

```

```

b.HasKey(m => m.Id);

b.HasOne(m => m.Chat).WithMany(c => c.Messages).HasForeignKey(m => m.ChatId).OnDelete(DeleteBehavior.Cascade);

b.HasOne(m => m.Sender).WithMany(u => u.Messages).HasForeignKey(m => m.SenderId).OnDelete(DeleteBehavior.Restrict);

});

```

```

modelBuilder.Entity<Call>(b =>
{
    b.HasKey(c => c.Id);

    b.HasOne(c => c.Chat).WithMany(ch => ch.Calls).HasForeignKey(c => c.ChatId);

    b.HasOne(c => c.Initiator).WithMany(u => u.Calls).HasForeignKey(c => c.InitiatorId).OnDelete(DeleteBehavior.SetNull);

});

}

}

using ChatApi.Models;

```

```

namespace ChatApi.Repositories.Interfaces;

```

```

public interface IUserRepository
{
    Task<User?> GetByIdAsync(long id);

    Task<User?> GetByUsernameAsync(string username);

    Task<IEnumerable<User>> GetAllAsync();

    Task<User> CreateAsync(User user);

    Task UpdateAsync(User user);

    Task DeleteAsync(long id);

}

using ChatApi.Models;

```

```

namespace ChatApi.Repositories.Interfaces;

```

```

public interface ISettingsRepository
{
    Task<UserSettings?> GetByUserIdAsync(long userId);

    Task<UserSettings> CreateOrUpdateAsync(UserSettings settings);
}

```

```

    }

    using ChatApi.Models;

    namespace ChatApi.Repositories.Interfaces;

    public interface IChatRepository
    {
        Task<Chat?> GetByIdAsync(long id);
        Task<IEnumerable<Chat>> GetAllAsync();
        Task<Chat> CreateAsync(Chat chat, IEnumerable<long>? participantUserIds = null);
        Task AddUserToChatAsync(long chatId, long userId, string? role = null);
    }

    using ChatApi.Models;

    namespace ChatApi.Repositories.Interfaces;

    public interface ICallRepository
    {
        Task<Call?> GetByIdAsync(long id);
        Task<Call> CreateAsync(Call call);
        Task<IEnumerable<Call>> GetByChatIdAsync(long chatId);
    }

    using ChatApi.Models;

    namespace ChatApi.Repositories.Interfaces;

    public interface IMessageRepository
    {
        Task<Message?> GetByIdAsync(long id);
        Task<IEnumerable<Message>> GetByChatIdAsync(long chatId, int limit = 50);
        Task<Message> CreateAsync(Message message);
    }

    using ChatApi.Data;
    using ChatApi.Models;
    using ChatApi.Repositories.Interfaces;
    using Microsoft.EntityFrameworkCore;

```

```
namespace ChatApi.Repositories.Implementations;
```

```
public class UserRepository : IUserRepository
```

```
{
```

```
    private readonly AppDbContext _db;
```

```
    public UserRepository(AppDbContext db) => _db = db;
```

```
    public async Task<User?> GetByIdAsync(long id) => await _db.Users.Include(u => u.Settings).FirstOrDefaultAsync(u => u.Id == id);
```

```
    public async Task<User?> GetByUsernameAsync(string username) => await _db.Users.Include(u => u.Settings).FirstOrDefaultAsync(u => u.Username == username);
```

```
    public async Task<IEnumerable<User>> GetAllAsync() => await _db.Users.ToListAsync();
```

```
    public async Task<User> CreateAsync(User user)
```

```
{
```


Контрольні запитання

1. Що собою становить діаграма розгортання?

Діаграма розгортання (Deployment Diagram) у UML показує, як система реалізується у фізичному середовищі. Вона відображає розміщення програмних компонентів на апаратних пристроях – наприклад, клієнт, сервер, база даних – і показує, як між ними передаються дані.

2. Які бувають види вузлів на діаграмі розгортання?

Фізичні вузли (Node): реальні пристрої, як-от сервер або комп'ютер.

Вузли виконання (Execution Environment): середовища, у яких запускається програмне забезпечення (наприклад, операційна система або вебсервер).

3. Які бувають зв'язки на діаграмі розгортання?

Асоціація (Communication Path): показує з'єднання та обмін даними між вузлами.

Залежність (Dependency): відображає, що один вузол або компонент залежить від іншого.

4. Які елементи присутні на діаграмі компонентів?

Компоненти: окремі частини програмного забезпечення.

Інтерфейси: точки взаємодії між компонентами.

Пакети: групування компонентів за логікою.

Зв'язки: показують взаємозалежності між компонентами

5. Що становлять собою зв'язки на діаграмі компонентів?

Вони показують, як компоненти залежать один від одного — який компонент використовує інший або взаємодіє з його інтерфейсом.

6. Які бувають види діаграм взаємодії?

Діаграма послідовностей – показує порядок викликів між об'єктами.

Діаграма комунікацій – показує зв'язки між об'єктами.

Діаграма часу – показує зміну станів у часі.

Діаграма огляду взаємодій – узагальнює кілька діаграм взаємодії

7. Для чого призначена діаграма послідовностей?

Вона описує порядок обміну повідомленнями між об'єктами у межах певного сценарію роботи системи.

8. Які ключові елементи можуть бути на діаграмі послідовностей?

Актори або об'єкти (Lifelines)

Повідомлення (Messages)

Смуги активності (Activation Bars)

Події створення або завершення об'єктів.

9. Як діаграми послідовностей пов'язані з діаграмами варіантів використання?

Кожна діаграма послідовностей деталізує певний варіант використання, показуючи, як саме виконується сценарій крок за кроком.

10. Як діаграми послідовностей пов'язані з діаграмами класів?

Об'єкти на діаграмі послідовностей є екземплярами класів, а повідомлення відповідають методам із діаграми класів.

Висновок

У ході виконання лабораторної роботи було закріплено практичні навички побудови діаграм, що відображають архітектурну структуру та логіку взаємодії компонентів програмної системи. Було спроектовано діаграму компонентів, яка показує структуру програмного забезпечення та

взаємозв'язки між основними модулями; діаграму розгортання, що ілюструє фізичне розміщення компонентів на апаратних вузлах; а також дві діаграми послідовностей, які демонструють обмін повідомленнями між об'єктами під час виконання сценарію системи.